Algorithms and data structures
MEMORY

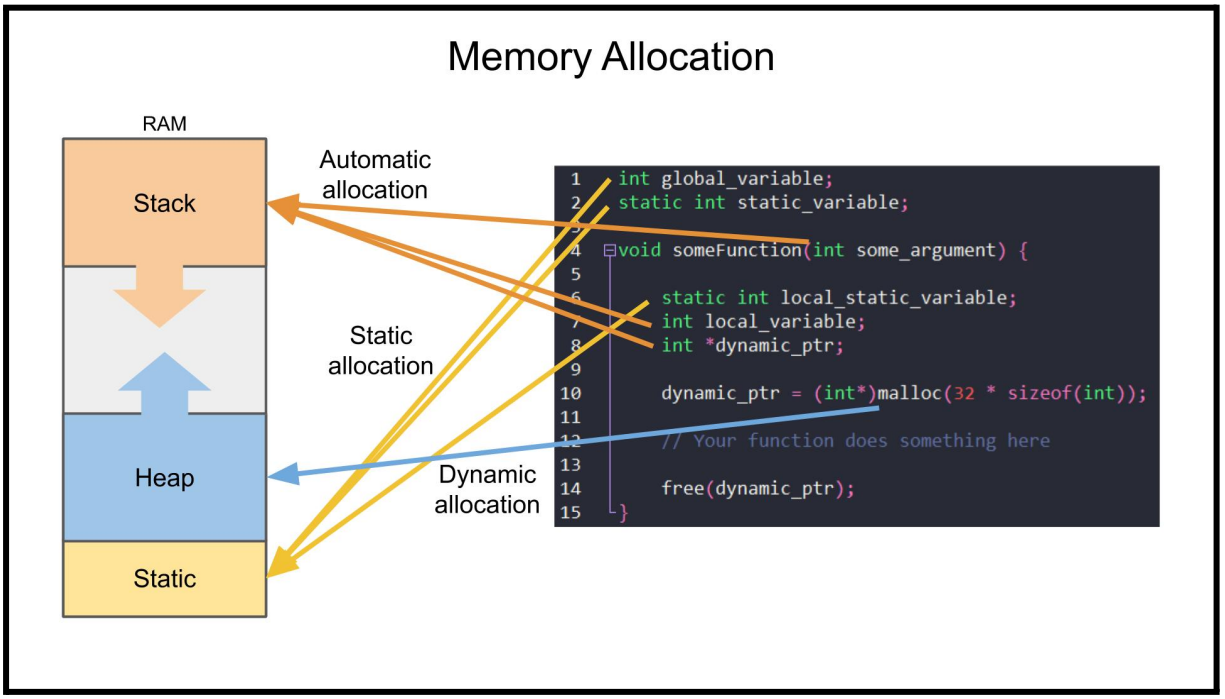


Fig. 01

Subject name: Algorithms and data structures
Associated module: 2PVG MA
Professor: Iván Sancho
Course: 2023/2024
Author: Sergio Madaleno y Fede Sanjuan

Index

1.- Memory stack explication **3.**

2.- Bibliography **5.**

1.- Memory stack explication

The memory stack is a fundamental data structure in programming and is used to manage program execution, particularly in the implementation of function calls and memory management. It functions as a fixed-size memory region and is typically organized as a Last In, First Out (LIFO) data structure.

Operations of a Memory Stack:

- Push: When a function is called, the local data of that function, such as local variables and relevant register values, is stored on the stack. This is achieved by pushing these data onto the top of the stack. The push operation increases the stack pointer to point to the newly stacked element.

- Pop: When the function finishes its execution, the local data stored on the stack is removed using the pop operation. This involves removing the top element from the stack and decreasing the stack pointer to point to the next element in the stack.

- Access to the top of the stack: Functions can access the data stored at the top of the stack. This is crucial for reading and modifying local variables and other data of the current function.

- Stack pointer management: A stack pointer is used to track the current location in the stack. When push or pop operations are performed, the stack pointer is updated to reflect the top of the stack.

Implementation of Function Calls:

The memory stack plays a critical role in the implementation of function calls in a program. Here's a description of how the stack is used in this context:

- When a function is called, a push operation is performed on the stack. This involves stacking the current state of the program, including the return value, registers, and local variables of the calling function. Then, a new stack frame is set up for the called function, which includes space for its own local variables and registers.

- The local variables and registers of the current function are stored at the top of the stack, and the function can access them through the stack pointer. The stack continues to grow as more nested function calls are made.

- When a function finishes its execution, a pop operation is performed on the stack. This involves removing the stack frame of the called function and restoring the program state to the calling function, including the return value.

Proper stack management is essential to avoid stack overflows or underflows, which could result in serious errors or program crashes.

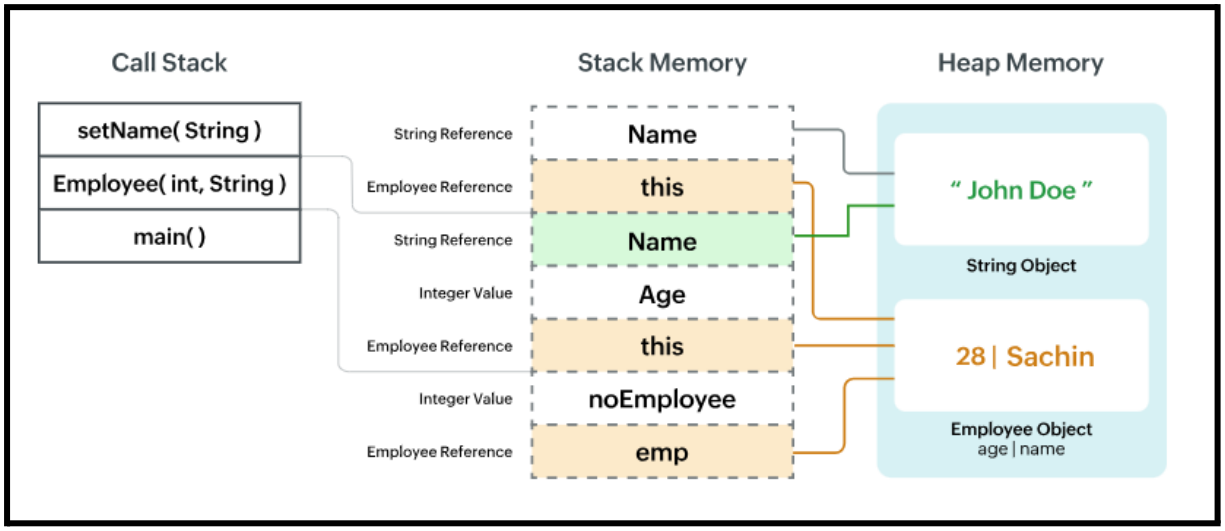


Fig. 02

The stack memory management depends on the references used in the code. For example, method calls are added on top of each other and removed when they exit the method's execution.

The portion of memory allocated for this purpose functions like a stack, and this is called the call stack. Similar to a call stack, stack memory is used for various temporary components, such as references to an object and temporary results.

2.- Bibliography

2.1 List of figures:

- Fig.01: HYME, Shawn. Introduction to RTOS - Solution to Part 4 (Memory Management) [online]. [Consulted on 11/28/2023]. Available in: https://www.digikey.com/en/maker/projects/introduction-to-rtos-solution-to-part-4-memory-management/6d4dfcaa1ff84f57a2098da8e6401d9c

- Fig.02: MAMAGEENGINE SITE24x7. Understanding heap and stack memory management [online]. [Consulted on 11/28/2023]. Available in: https://www.site24x7.com/learn/java/heap-and-stack-memory-management.html

2.2 Bibliography:

- HEKTOR DOCS. Gestión de la memoria (Stack y Heap) en C++. [online]. [Consulted on 11/28/2023]. Available in: https://docs.hektorprofe.net/cpp/07-punteros-memoria/06-tipos-memoria-stack-heap/